Rahul Rajkumar. NetID: rr1437

This ship is on fire!

Quick disclaimer: Initially, when I started this project, I wanted to write in C++ because I was learning the language at the time and thought applying it to this project might be a good way to practice. However, when I started coding this project, an important thought later occurred to me. I don't know enough C++. Clearly, that was an important thought. So I pivoted from using C++ to writing all the code in Python. So, while I told the TA I would do this project in C++, I actually did it in Python. So I apologize for any confusion. However, the underlying architecture of the project is still the same as what I claimed it would be.

Introduction

Before I talk about the algorithm I used for bot 4, I would like to talk about the implementation of this project. The ship is a D x D grid of "Tile" objects. Currently it's in a list format, however as I was reviewing the code before writing this paper, I realized it would've been more efficient to use Numpy 2d arrays instead of lists, since I use the package anyway for generating random seeds, I hope to change this in a future project, and write code more efficiently such as using parallelization to increase how fast this program runs so experiments can be done faster. These tiles (which are basically the cells in the ship) have the ability to track whether the tile is open or closed, whether it is currently on fire, whether or not it is currently holding the bot or the button, etc. The tile objects do this by having attributes, such as has_bot, on_fire, etc. Then we portray a fire or bot movement by setting the current tiles attribute to false and setting the neighboring tile's attribute to true. As per the instructions, the code sequentially generates the ship by first, building out the grid of tiles, randomly opens one of them, opens up other cells, and finally opens up at least half of the dead end cells. Importantly, the ship is also generated using a seed, so if you would like to generate the same ship again, you would just use the same seed.

Following the TA's advice, I avoided the mistake of letting newly burning cells spread fire again in the same step. To spread the fire,first we would need to decide which cells will actually catch on fire using only the fire cells that were already burning at the current time step, so a cell that is supposed to catch on fire shouldn't flip to a burning cell and impact the probabilities of neighboring cells catching on fire. The fire itself also uses a random seed like the ship generation. This is useful because every bot on a trail can face the exact same fire and the way it spreads, keeping things fair to compare the performance of each bot for every trial. The program also records why a bot might fail, the reasons include: it walked into the fire, the fire spread onto it, the button burned first, or it got cut off because every route to the button runs through fire.

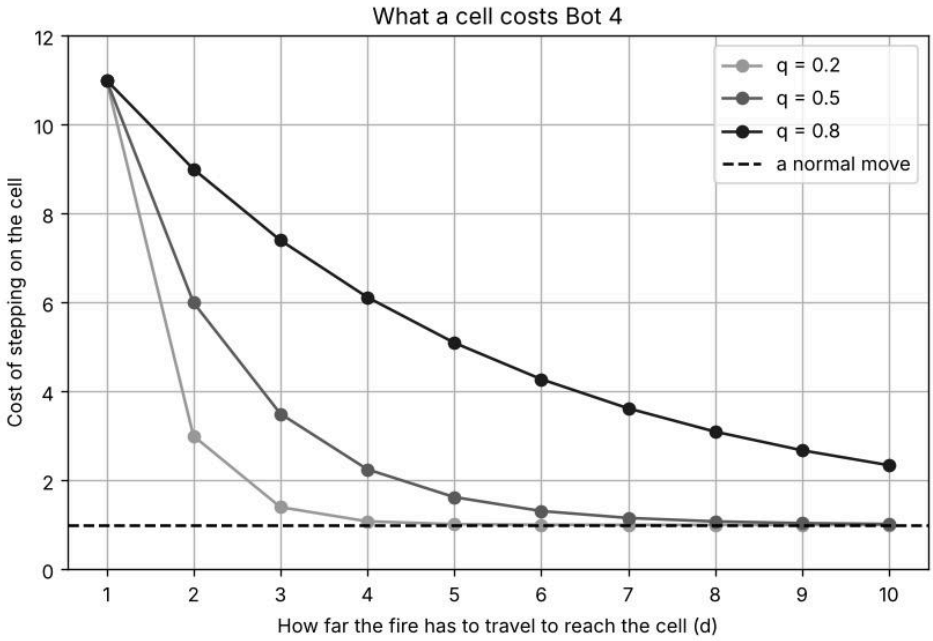
Now for the search algorithms. Because bots 1-3 require the shortest path, the best algorithm to use would be Breadth First Search (BFS). For bot 1, BFS would run once, finding the shortest path at the initial time step from the bot's current location to the button, avoiding only the fire cell that is currently burning, and ignoring the spread of the fire. For bot 2, BFS runs at every single time step so it can account for the spread of fire. Bot 3, runs a very similar set up to bot 2, however the cells that are restricted that BFS should avoid also includes the neighbors of a cells that are on fire, if no such path exists with those restrictions then bot 3 just falls back to bot 2's every timestamp updating BFS algorithm to reach the button.

Bot 4 Algorithm

For bot 4, I was leaning towards using some version of A* to find an optimal shortest path to button while avoiding the fire cells. In one of our recitations, the TA had mentioned that a common heuristic for A* could be using euclidean distance measurements to gauge how far away you are from your goal node. In this case, since we are working with grids, a better heuristic might be to calculate the Manhattan distance from the bot to the button. In addition to this heuristic, I also wanted to add a cost component to paths that would penalize its score if it tried to persuade the bot into taking a path where it might meet a fire cell. The obvious hypothesis was, cells next to the fire are the ones that are most likely to burn next, and the faster the fire (higher q value), the further that danger spreads. So, in order to understand this dynamic, bot 4 makes a heat map of the ship, where the closer a cell is to a fire, the warmer it is, and the costlier it is to step on or use. Hence, in addition to the Manhattan distance, A* also takes into account the cost of using a cell to find the cheapest path to the button. The main idea is, bot 4 goes around the hot area when the way around isn't too long, and goes through it when every other way is much further away.

At each time step bot 4 does the following things:

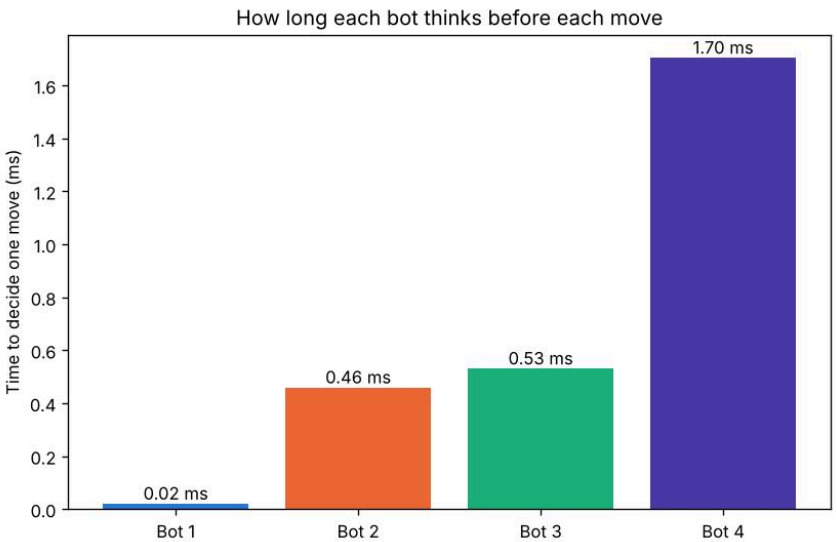
1. It runs BFS that starts from every burning cell at time step t at once and only moves through open cells, which gives us d, the number of cells the fire has to travel to reach each cell. This could also be thought of as a distance map, where we take a few starting cells or the cells that are currently on fire and calculate how far away or the number of cells we had to travel to reach the current cell we land on.
2. The temperature of each cell is calculated using this formula: heat = q^(d-1), where d is the distance from the closest fire cell, calculated from the previous step. So a cell right next to the fire has a temperature of 1, and each cell further away multiplies the heat by q, because for the fire to eventually reach that cell, it has to get lucky with a chance of q, once for every cell on the way. With a high q or fast spreading fire, cells tend to be warmer but for a slow spread fire or low q, the temperature dies off relatively quickly.
3. Once the temperature of each cell is calculated or when the heat map is constructed, bot 4 sets the cost of stepping on each cell. The formula for that is: (1 + 10 * heat) or 1 million for a burning cell. The extremely large cost of a burning cell is to persuade the bot to never step on a burning cell as every route that avoids the fire costs far less, so A* only goes through the fire if there is no way around it, which, at that point, bot 4 understands that there is no path to the button.
4. After we have calculated the costs, bot 4 runs A* to the button. A* always takes off a cell from the fringe with the smallest g + h, where g is the cost of getting there so far and h is the Manhattan distance from to the button. If going through that cell is the cheapest way found so far to one of its neighbors, it saves the new cost and prev and adds the neighbor to the fringe. When the goal is reached or the button node comes off the fringe, it follows prev back to the start node to reconstruct the path it took.



This graph shows the cost bot 4 assigns each cell which is based on how far the fire has to travel to reach that particular cell. As one may suspect, costs tend to be much higher for higher values of q, the difference in cost between d (or distance from fire cells) is also smaller compared to lower values of q. E.g. for q=0.8, the difference in cost between d=1 and d=3 is around 4, while same difference in cost when q is 0.2 is around 10.

One thing I want to emphasize is that the heat map and the Manhattan distance metrics are for two different purposes. The heat map exists to calculate the costs of each step on each cell closest and furthest from the fire cells, which decides which path is optimal, it also what differentiates bot 4 from bot 2 or bot 3's restrictions. The heuristic for A* doesn't really change the path A* finds because every step costs at least 1, so it's never more than the real cost left, and only exists to guide or persuade the bot to look at paths closer to the button first.

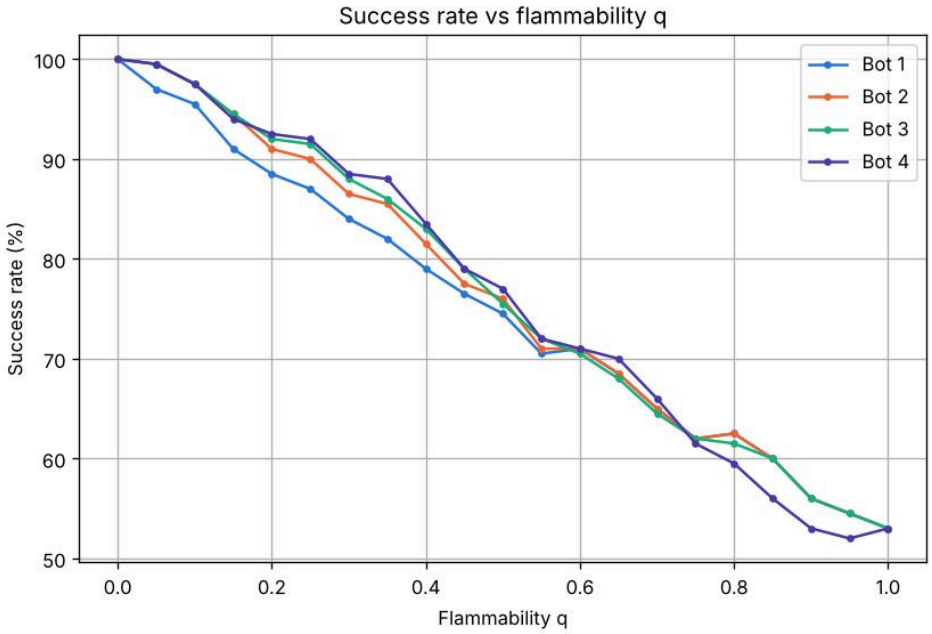
Also, for efficiency, each tile's open neighbors are already worked when the ship is built, so the searches never have to check for walls. Every step, bot 4 only needs to run BFS once, one pass over the ship and run A* once. All four bots reuse the same ship on a trial, and a run stops as soon as the bot wins or fails.



The bar chart on the left hand side shows how long each bot takes on average to decide one move. Bot 1 only plans once at the start, so it barely needs to think at all so it has the lowest thinking time. While bot 4 takes about 3 to 4 times longer than bots 2 and 3, because it has to run BFS from the fire (build the distance map), builds the heat map and then run finally runs A* at every time step, instead running just BFS.

Experiments

For the experiments, I used D=50 for the main run, because a trial on a 100 by 100 ship takes about 10 times longer. I also tried running experiments with D=25 and 100 separately. I tried every q from 0 to 1 in steps of 0.05, with 200 trials at each q value, so a total of 21 steps since I'm including both ends, summing up to a total of 4.2k trials, with all four bots on every one. The seed of the ship is tied to the iteration, so for trial i, the ship seed would be i, so the every q had the same ship seed and the only that actually changes from q to the next q is how fast the fire spreads, as intended. Also because there is a fire seed, we can directly compare the bots since the difference between the two bots are originated by the bots decisions.



This graph shows the success rate against flammability q for every bot, with 200 trials at each q

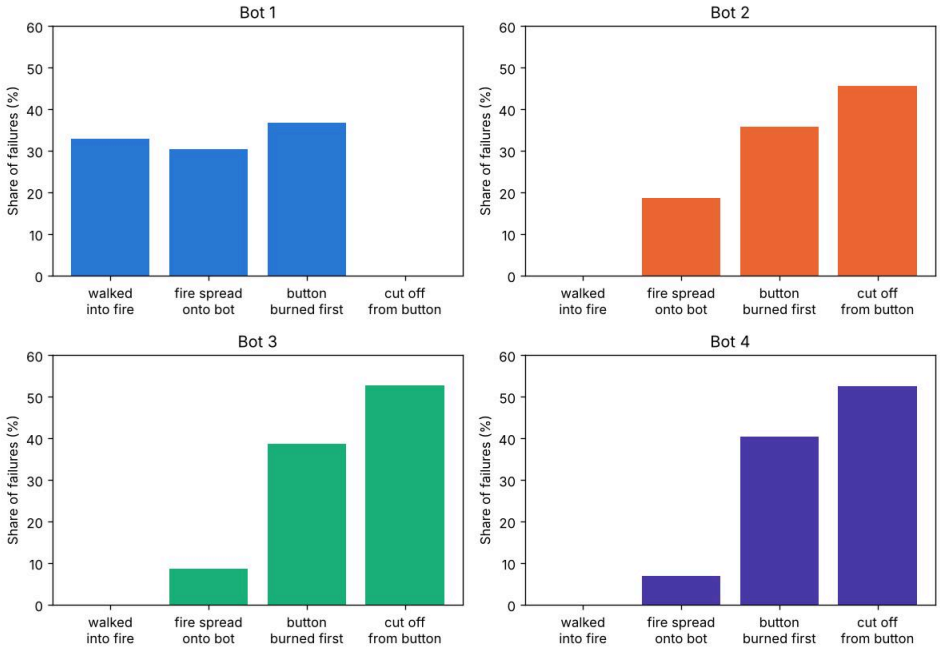
q	Bot 1	Bot 2	Bot 3	Bot 4
0.1	95.5%	97.5%	97.5%	97.5%
0.2	88.5%	91.0%	92.0%	92.5%
0.3	84.0%	86.5%	88.0%	88.5%
0.4	79.0%	81.5%	83.0%	83.5%
0.5	74.5%	76.0%	75.5%	77.0%
0.6	71.0%	71.0%	70.5%	71.0%
0.7	65.0%	65.0%	64.5%	66.0%
0.8	62.5%	62.5%	61.5%	59.5%
0.9	56.0%	56.0%	56.0%	53.0%
1	53.0%	53.0%	53.0%	53.0%

The table to the left shows the success rate for each bot based on the value of q. For very small q values all the bots are about equally good (at q=0 they all win every trial - which makes sense since the likely chance of another open cell catching on fire is extremely low), and for large q they are equally bad (it's hard to escape hard fires). However there are some interesting observations to take a closer look at. From q=0.6 up, Bot 1, which never replans, wins exactly the same trials as Bot 2, and at q=1 all four bots win exactly the same 106 of 200 trials (53%). Because around those ranges the fire then moves as fast as the bot, the only thing that really matters is whether the bot started closer to the button than the fire.

q range	vs Bot 1	vs Bot 2	vs Bot 3
0.1 to 0.2	+3.0	+0.3	+0.0
0.25 to 0.65	+3.1	+1.5	+0.8
0.7 to 1	−1.7	−1.7	−1.5

I also wanted to show difference in success rates between a range of q values. As you can see, between 0.1-0.65, bot 4 matches or has higher success rate than the other bots, however for ranges higher than 0.7, bot 4 does worse. This makes sense, for lower q values the heat map or the temperature among cells is distributed in a neat fashion in which the bot can use to make smart decisions, however in larger q values, the heat map is skewed and the cost among cells is very similar so the information about the cost of each cell provides less value.

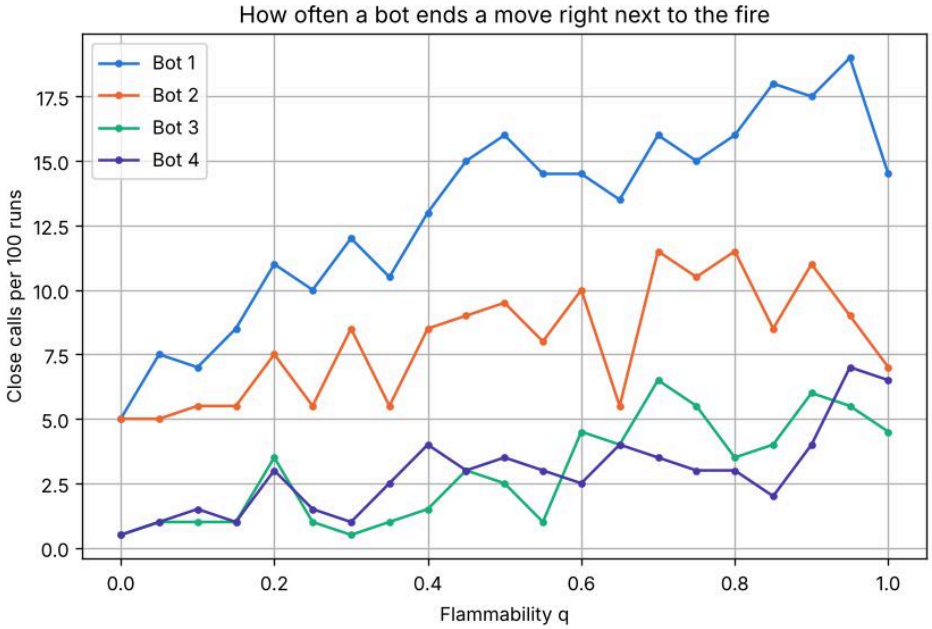
Bot Failure



This bar charts shows the frequency of failures for each bot split by the category of reasons

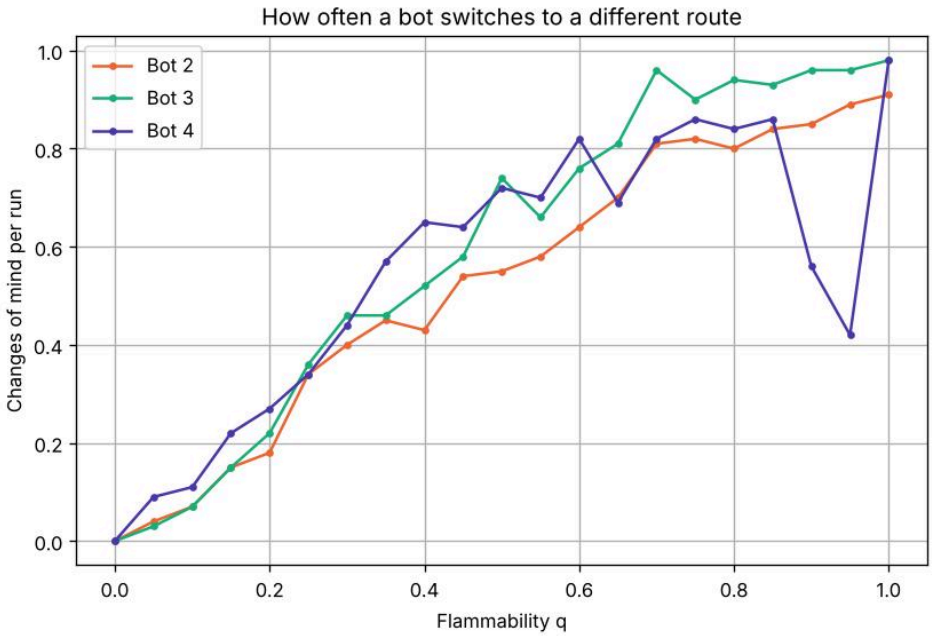
Basically, almost a third of bot 1's failures comes from walking into the fire, which no other bot ever does, because bot 1 never actually looks again after planning so it there's a possibility that it actually walked into the fire. Bot 2 walks along the edge of the fire, so around 20% of its failures are the fire eventually spreading onto it. That metric is much lower for bots 3 and 4. For bots 2-4, almost all of failures are aggregated among the button burning first and the bot getting stuck.

To make some interesting observations I also counted two things during every run. One of things I kept track of is a variable called "close calls". The idea is that the moves that end right beside a burning cell is a close call because there is a chance that cell it's currently on will catch on fire. The graph below show that bots 3 and 4 have the least amount of close calls because they are actively trying to find a path to avoid the fire. It is in per runs because close calls tend to be rare and it is hard to interpret the number of close calls per 100 runs.



Line chart shows how often a bot ended up being close to a fire cell

The second metric I also collected is a something called “change plan”. It counts the number of times a bot deviates from it’s previous plan. Every bot that replans on every time step tends to change it’s mind more often as the fire gets faster or q is larger, which is intuitive in some sense because as the fire spreads faster, it blocks more routes, so there is a bigger need to change course of direction. Because bot 4 has more information to rely on, it changes it mind more often than bot 2 around a large range of q values. One thing I found super interesting was the dip in changes of plan around the q value of 0.9, I believe around that value, the heat barely fades with distance, so a longer route is rarely worth its extra steps and bot 4 mostly stops switching to detours.



Line chart shows how often a bot ended up changing its plan from its previous

The original point of these metrics was to collect information to check whether if there was a better decision that the bots could’ve made when they failed. For bots 1 and 2, the close calls somewhat bolster the argument that staying at least 1 cell further away from the fire helps reducing the failure rate, which is what bots 3 and 4 try to do. For improving bot 4, the better choice it should’ve made would’ve been to, at a high q, to just run straight for the button. Instead it currently tries to avoid the fire and takes really long routes, which gives the the fast pace fire to ability to catch up to it and fail the bot.

I want to mention that I went through a few iterations of bot 4 before building this one. Initially I want to mark any cell that had higher than a 60% of catching on fire to be marked as if they were already on fire, however this ended up being extremely similar to bot 3’s restriction where there is a buffer around the existing fire cell. Then I tried to forecast the path of the fire and see if there is time step in which the fire and the bot meet and that route would be prevented, however this brought too much complexity into building an algorithm and ruined the whole purpose of building an cost method for A* that solves a relaxed form of an existing problem.

The Ideal Bot

I think the ideal bot would be something very difficult to build. But hypothetically, the bot would pick a route that maximizes its chances of pressing the button, while simultaneously (with less effort than trying to reach the button) trying to avoid the fire. I think one of thing that the ideal bot would do (that I had failed to build) is try to predict or forecast the route of the fire. Because if it had information on where the fire would be in t+5 time steps, then it would take the route that might seem bad in the short term, maybe it moves away from the button in the first 3 time steps, but avoid reaching the fire in the long term, so there the path must be time dependent.

One way of forecasting this correctly would be to look at all the possibilities that the fire might spread, pick the top 3-5 routes that are most likely to happen and assume those open cells will be burning at whatever time step it is forecasted through. In terms of computation, considering my bot 4 takes significantly much more time than any of the other bots, it's clear that trying to build a plan at every time step becomes computationally heavy. Maybe a smarter way to reduce computation while still trying to be "intelligent" would be to decide when it's important to replan versus when we should just blindly continue with our existing plan. For example, instead of doing a search at every time step, if we are forecasting, and we know that the bot will be continue to be far away from the fire for the next 3 time steps, then we shouldn't be replanning until the 3rd time step, maybe it's even easier to fixed amount of times you can forecast or replan the route, like every 5 steps or so.

To answer when might it be intelligent to just make a break for it and hope, I would say during the extreme values of q. When q is really low, the chance of fire spreading is so low, it just makes sense to run simple BFS once. When q is really high then it's pretty obvious the chances of the bot surviving no matter how much information it takes is so low that it doesn't make sense to use all that effort. This idea makes sense because the data shows that all four bots preform nearly identical for these extreme values, at which point, you should just run basic BFS once and hope for the best.

Reflection

I do want to state how I felt about this project. Overall I believe this was a great project to apply some of the search algorithms. Initially I was bit lost understanding the priority function of A* with the function g(n) + h(n) since I incorrectly understood that A* only used the heuristic to rank the priority queue but then when I was coding it following the typed notes, then I understood where the g(n) came from. One thing I wish I did better was how efficient I wrote this code, I am sure there is faster way to build the grid than making a 2d array of lists, because of this I was only able to run a fewer amount of trials. I also liked the visualizations these metrics produced since they really showed the difference in performance and behaviors for each of the bots. I am part of a student-led club here at Rutgers named Rutgers Economic Labs (https://www.rutgerseconomics.org/), where we partner with government agencies to use data and other analytical information to provide valuable research. Because those projects tend to be more quantitative, I build a lot of graphs using matplotllib for analysis, but this is the first time I've applied those skills outside of economic or financial data and to computer science related project so I found that very interesting.